

《C#程序设计及应用》大报告

题目：工控上位机监控系统

专业年级：自动化 232

学号：202310232052

姓名：章家铭

指导教师：虞旦

2025 年 12 月 27 日

[题目]

工控上位机监控系统

[概述]

1. 选题背景与意义

在现代工业自动化与嵌入式系统开发领域（如无人机飞控调试、电机驱动器研发），研发人员面临着巨大的数据观测挑战。传统的开发模式通常依赖于通用的串口调试助手（如 SSCOM、Putty）来查看下位机输出的调试信息。然而，这些工具大多仅能以原始 ASCII 文本流的形式展示数据，例如屏幕上飞速滚动的 TEMP=45;V=12;RPM=3000。[1]在设备高速运转时，参数变化极快，单纯依靠肉眼观察文本流，很难捕捉到电压的瞬时跌落、温度的异常攀升或转速的微小波动，而这些细节往往是判断系统稳定性的关键。此外，传统工具缺乏历史数据的持久化存储与回溯分析功能，且必须要求工程师驻守在设备旁进行“人肉监控”，无法满足远程无人值守测试的需求。[2]

为了解决上述痛点，本项目结合《C#程序设计及应用》课程所学知识，综合运用 Windows 桌面开发（WinForms）与 Web 开发（ASP.NET Core）技术，设计并实现了一套“基于 C#的工控上位机监控系统”。该系统旨在为嵌入式开发人员提供一个低成本、高效率的调试“利器”，实现了从数据采集、解析、实时可视化到云端远程监控的全链路覆盖，具有显著的工程应用价值。

2. 程序功能

本系统围绕“数据流”的生命周期，实现了以下四大核心功能：

- 高频数据采集与智能解析：系统底层基于串口通信机制，能够接收来自下位机（或模拟器）发送的高频 ASCII 数据流。内置的解析引擎具备强大的容错能力，能够自动清洗脏数据，并将非结构化的文本实时转换为结构化的 C# 对象。
- 实时可视化状态监控：摒弃了枯燥的文本显示，桌面端采用了直观的图形化界面。通过 GDI+ 技术自定义绘制的实时波形图，能够以毫秒级的响应速度描绘出电压与转速的变化趋势；拟物化的仪表盘控件则直观展示了当前系统的负载情况。
- 多级报警与日志审计：系统内置了“安全包络线”检测机制。当采集到的温度、电压等关键指标超出预设的安全阈值时，UI 界面会立即通过颜色翻转（如变红）进行视觉报警，并将异常事件记录至本地 SQLite 数据库，便于后续的故障复盘。
- 远程 Web 看板：打破了物理空间的限制，系统集成了一个轻量级的

Web 服务器。管理人员或导师无需安装任何客户端，只需通过浏览器登录 Web 页面，即可实时查看实验室设备的运行状态及历史数据列表，真正实现了“口袋里的监控室”。

3. 系统架构

为了兼顾“现场控制的实时性”与“远程访问的便捷性”，本系统创新性地采用了 C/S (WinForms) + B/S (Web) 的混合架构模式，并引入了“边缘计算”的设计思想。[3]

- 桌面端 (Client - 边缘节点)：作为系统的核心控制台，它直接与硬件设备交互。WinForms 框架的低延迟特性使其能够轻松应对 10Hz 甚至更高频率的数据刷新，负责完成数据的清洗、校验、本地存储以及对硬件的反向控制（如急停指令下发）。[4]
- Web 端 (Server - 云端展示)：作为纯粹的数据展示层，它不直接连接硬件，而是从共享的数据中心读取信息。[5]采用 ASP.NET Core MVC 架构，利用 Bootstrap 前端框架实现响应式布局，确保在 PC、平板和手机上均有良好的访问体验。[7,8]
- 数据层 (Data Center)：这是连接 C/S 与 B/S 架构的桥梁。[6]两者通过共享同一个 SQLite 数据库文件进行通信。为了解决文件型数据库在多进程访问时的锁冲突问题，系统开启了 WAL (Write-Ahead Logging) 预写日志模式，实现了真正意义上的并发读写。[9]

[框架平台]

1. 开发环境
- 操作系统：Windows 11
- 集成开发环境 (IDE)：Microsoft Visual Studio 2026 Community

2. 软件框架与技术选型
- 桌面端：.NET Framework 4.8 (Windows Forms)
  - 选型理由：虽然 WPF 技术更新，但在工控领域，WinForms 凭借其极高的成熟度、对老旧硬件的良好兼容性以及简单直接的 GDI+ 绘图接口，依然是上位机开发的首选。本项目使用了第三方控件库 System.Windows.Forms.Ribbon 来构建现代化的 Office 风格菜单，提升了软件的交互体验。
- Web 端：ASP.NET Core 10 MVC
  - 选型理由：ASP.NET Core 是微软推出的跨平台、高性能 Web 框架。其内置的依赖注入 (DI) 容器非常适合管理数据库服务，且 MVC (Model-View-Controller) 模式能够清晰地分离业务逻辑与页面展示，便于后期维护。前端采用 Bootstrap 5 + jQuery，以最小的代码量实现了美观的响应式界面。
- 数据库：SQLite (System.Data.SQLite)
  - 选型理由：对于单机版的工控软件，部署 SQL Server 或 MySQL 显得过于笨重且配置复杂。SQLite 作为一个“单文件”数据库，具备

零配置、易迁移（直接复制文件）的特点。特别是其 WAL 模式，完美解决了本项目中“桌面端写、Web 端读”的并发需求。[10]

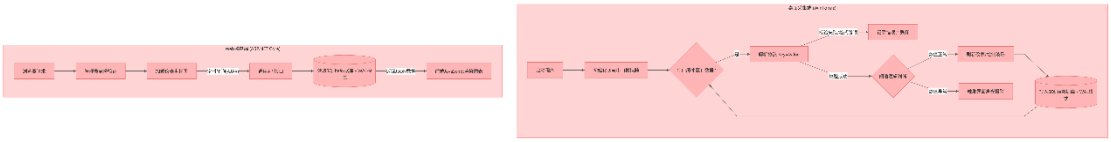
[软件设计]

程序流程图与数据流向分析：

本系统的逻辑设计遵循“生产者-消费者”模型。桌面端作为数据的“生产者”，源源不断地从串口获取原始数据并“生产”出结构化记录存入数据库；Web 端作为“消费者”，定时从数据库中“消费”数据并展示给用户。

具体流程逻辑如下：

1. 初始化阶段：程序启动，加载 Ribbon UI，初始化 SQLite 数据库连接池，并开启后台监听线程。
2. 采集循环：
   - 监听串口缓冲区，当检测到换行符时截取一帧数据。
   - 传入解析器进行校验，若格式非法（如校验和错误或关键字段缺失）则丢弃并记录错误日志。
   - 若格式合法，实例化为 C# 对象。
3. 业务处理：
   - 阈值判断：将当前值与 App.config 中的阈值配置进行比对。
   - UI 分发：根据判断结果，更新 UI 状态（正常绿色/报警红色），并触发 GDI+ 重绘事件更新波形。
   - 持久化：将对象序列化并写入 SQLite 数据库。
4. Web 响应：
   - Web 服务器接收前端 AJAX 请求。
   - 查询数据库最近 N 条记录。
   - 返回 JSON 数据包供前端渲染。



图一：流程图

2. 数据库设计说明

数据库设计遵循“够用且高效”的原则。考虑到工控数据的高频写入特性（可能达到 10Hz 以上），表结构设计尽量精简，且必须包含时间维度索引。

本系统核心数据表名为 `TelemetrySamples`，用于存储所有采集到的遥测数据。该表的设计包含了五个关键字段：首先是 Id 字段，采用整型（INTEGER）并设置为自增主键，既作为数据的唯一标识符，也能保证数据的物理存储顺序；其次是 Timestamp 字段，存储精确到毫秒的采集时间（DATETIME），这是查询优化的核心——通过在此字段建立索引，极大地提升了 Web 端“查询最近 N 条”或“按时间段筛选”时的响应速度；MetricName 字段用于记录物理量名称（如 Motor_RPM），以此区分不同的传感器数据源；`Value` 字段存储具体的监测数值，

选用双精度浮点数（REAL）以确保工业测量所需的精度；最后是 Unit 字段，用于存储单位（如 V, RPM），作为辅助信息传给前端展示，避免了硬编码带来的灵活性缺失。

3. 软件业务逻辑说明

1. 鲁棒的协议解析逻辑

在工业现场，电磁干扰可能导致串口数据出现乱码（如 TEMP=4#5）。为了保证软件不崩溃，本系统实现了一套“宽容”的解析算法：

- 定界符切分：首先利用 ；将数据帧切分为多个 KV 段。
- 结构验证：检查每个段是否严格包含一个 = 号。
- 类型安全转换：使用 double.TryParse 替代 double.Parse。前者在转换失败时只会返回 false 而不会抛出异常，从而保证了上位机在接收到脏数据时依然能稳定运行，仅丢弃坏点数据。

2. 高并发下的读写分离逻辑

这是本系统最核心的技术难点。普通的 SQLite 模式在 WinForms 写入数据时会锁死整个文件，导致 Web 端读取超时报错 Database is locked。

- 解决方案：启用 WAL (Write-Ahead Logging) 模式。
- 原理：在 WAL 模式下，WinForms 的写操作不再直接修改主数据库文件（.db），而是追加写入到一个临时的日志文件（.db-wal）中。这使得写操作不会阻塞读操作。Web 端的读操作可以同时读取主文件和日志文件中的内容。SQLite 引擎会在合适的时机（Checkpoint）自动将日志合并回主文件。这一机制完美支撑了混合架构的数据同步。

3. GDI+ 双缓冲绘图逻辑

为了实现流畅的实时波形图，系统没有使用沉重的 Chart 控件，而是直接调用 GDI+ 接口。

- 坐标变换：将数据的“时间-数值”逻辑坐标映射到 PictureBox 的“像素”物理坐标。
- 双缓冲技术：为了防止高频刷新（如 10Hz）导致的屏幕闪烁，系统在内存中创建一张位图（Bitmap），先将所有线条、网格、文字绘制在内存位图中，最后一次性将整张位图拷贝到屏幕上。

4. 软件使用说明

1. 环境准备：确保目标机器已安装 .NET 6.0 Runtime（用于 Web 端）和 .NET Framework 4.8（用于桌面端）。
2. 启动采集端：
   - 双击运行 task.exe。
   - 在顶部 Ribbon 菜单栏点击“连接设备”（系统默认连接内部模拟器生成数据）。
   - 点击“启动引擎”按钮，观察中间的数据列表开始滚动，右侧的电压波形图开始动态绘制。
   - 可尝试点击“急停”按钮，模拟设备故障，观察日志记录。
3. 启动监控端：
   - 保持桌面端运行（切勿关闭，否则数据库写入会停止）。

- 运行 Web 项目或在浏览器访问发布地址（如 https://localhost:7152）。
- 输入管理员账号（默认 admin / passwd）登录。
- 点击导航栏的“仪表盘”，即可看到与桌面端完全同步的实时数据表格，页面会自动每秒刷新一次。

4. 重要程序代码及页面功能说明

- 核心代码：数据解析服务 (TelemetryService.cs)

该方法体现了系统的核心解析算法，采用工厂模式思想，根据不同的 Key 生成对应的实体对象。

```
// 解析单行协议数据：TEMP=23.4;V=12.1;RPM=1500
public TelemetrySample ParseLine(string line)
{
   // 1. 数据清洗：去除空行和首尾空格
   if (string.IsNullOrWhiteSpace(line)) return null;
   // 2. 协议分割：按分号拆解为键值对数组
   var parts = line.Split(';');
   foreach (var part in parts)
   {
       var kv = part.Split('=');
       // 格式强校验：必须包含 Key 和 Value 两个部分
       if (kv.Length != 2) continue;
       string key = kv[0].Trim().ToUpper();
       // 3. 安全类型转换：使用 TryParse 防止程序因脏数据崩溃
       if (double.TryParse(kv[1], out double value))
       {
           // 根据 Key 封装为统一的 TelemetrySample 对象
           // 这里还可以扩展更多的传感器类型
           if (key == "RPM")
               return new TelemetrySample { MetricName =
"Motor_RPM", Value = value, Unit = "r/min", Timestamp = DateTime.Now };
           else if (key == "V")
               return new TelemetrySample { MetricName =
"Bus_Voltage", Value = value, Unit = "V", Timestamp = DateTime.Now };
       }
   }
   return null; // 若无法解析出有效数据，返回空
}
```

- 核心代码：Web 端数据接口 (DashboardController.cs)
  Web 端控制器展示了标准的 MVC 设计模式与依赖注入的使用。

```
public class DashboardController : Controller
{
    private readonly TelemetryQueryService _queryService;
    // 构造函数注入：通过 ASP.NET Core 的 DI 容器自动获取数据库服务实例
    // 避免了在 Controller 中手动 new 对象，降低了耦合度
    public DashboardController(TelemetryQueryService queryService)
    {
        _queryService = queryService;
    }
    [Authorize] // 权限控制特性：确保只有登录用户才能访问此接口
    public IActionResult Index()
    {
        // 性能优化：仅获取最近 50 条记录，防止全表扫描导致页面加载过慢
        // 这种分页/限制策略在处理海量工控数据时是必须的
        var data = _queryService.GetRecentSamples(50);
        return View(data); // 将数据模型传递给视图层渲染
    }
}
```

- 页面功能说明
  - WinForms 主界面：
    - 功能区：顶部采用了微软的 Ribbon 菜单，将功能划分为“连接”、“控制”、“查看”三个逻辑组，降低了用户的学习成本。
    - 实时仪表盘：利用 ProgressBar 控件巧妙模拟了转速负荷表，绿色代表正常，红色代表过载，直观易懂。
    - 动态波形图：右侧的黑色背景区域为自定义绘图区，利用 GDI+ 实时描点连线，展示电压的纹波稳定性，这是工程师分析电源质量的重要窗口。
  - Web 监控仪表盘：
    - 响应式表格：基于 Bootstrap 的 Grid 系统，表格在 PC 端显示完整信息，在手机端自动适配宽度。
    - 智能高亮：Razor 视图引擎在渲染 HTML 时会检查数值大小，自动为异常数据添加 text-danger CSS 类，使 Web 端用户也能第一时间发现警报。

[系统调试]

在将理论转化为代码的过程中，我遭遇了数次程序崩溃与逻辑错误。以下是

三个最具代表性的 Debug 过程记录：

1. SQLite 依赖项缺失问题
   - 问题现象：程序在我的开发机（安装了完整 VS 环境）上运行一切正常，但当我把编译好的 bin 文件夹复制到没有安装 VS 的实验室电脑上时，双击 exe 直接闪退，系统日志报错 System.DllNotFoundException: Unable to load DLL 'SQLite.Interop.dll'。
   - 排查过程：起初我以为是 .NET Framework 版本问题，重装后无效。后来通过查阅 StackOverflow，发现 System.Data.SQLite 是一个托管的 C# 壳，底层调用的是非托管的 C++ 库 (SQLite.Interop.dll)。这个 C++ 库严格区分 x86（32 位）和 x64（64 位）。我的开发机 VS 默认是 "Any CPU"，运行时自动适配了架构，但部署包里可能缺失了对应的原生 DLL。
   - 解决方法：在 VS 项目属性中，将“目标平台”从 "Any CPU" 明确强制设置为 x64，并重新通过 NuGet 还原包。编译后，检查输出目录，确认生成了 x64 文件夹且里面包含 SQLite.Interop.dll，问题解决。
2. 线程安全陷阱 —— 跨线程 UI 更新异常
   - 问题现象：串口数据的接收是在一个独立的后台线程中进行的。当我试图在 DataReceived 事件处理函数中直接写 lblVoltage.Text = value; 时，程序立即抛出异常：“线程间操作无效: 从不是创建控件的线程访问它”。
   - 排查过程：回顾 Windows 消息机制，我意识到 WinForms 的 UI 控件并不是线程安全的，只能由创建它的主线程（UI 线程）去更新。
   - 解决方法：使用了 Control.Invoke 机制。在赋值前先判断 InvokeRequired 属性，如果为真，则通过委托（Delegate）将更新操作“封送”回主线程执行。虽然代码变长了，但这保证了程序的绝对稳定。
3. 并发死锁 —— 数据库文件被锁定
   - 问题现象：当我在桌面端开启“高速采集模式”（10ms 一次写入）时，只要一刷新 Web 页面，WinForms 端就会卡顿，甚至抛出 SQLiteException: database is locked。
   - 排查过程：SQLite 默认是单线程写模式，且写操作需要独占锁。当 Web 端进行读取（尤其是复杂查询）时，会持有一个共享读锁，这阻止了 WinForms 获取写锁。高频写入加剧了这种冲突。
   - 解决方法：查阅 SQLite 官方文档关于并发的部分，发现了 WAL 模式。我在连接字符串中简单地添加了 ;Journal Mode=WAL;。启用后，写操作不再阻塞读操作，两端终于能够和谐共处，即便在每秒 100 次写入的压力下，Web 端依然能秒开。

[心得及体会]

这次《C#程序设计及应用》的课程设计，对我而言，绝不仅仅是一次代码的堆砌，而是一场从“写代码”到“做软件”的思维蜕变。

刚拿到题目时，我本以为这只是个简单的“增删改查”作业。我自信地认为，

凭我之前写控制台程序的经验，两三天就能搞定。然而，现实很快给了我一记重锤。当我第一次试图把 WinForms 采集的数据实时显示在 Web 页面上时，我才发现“数据共享”远没有我想象的那么简单。文件锁死、线程冲突、乱码解析……每一个问题都像拦路虎一样横亘在面前。

最让我印象深刻的是调试“数据库锁死”的那个晚上。那几天我只要一运行程序，两端就“打架”，互相报错。我尝试了各种“野路子”，比如用文件读写锁、甚至想过让 Web 端直接读内存映射文件，但效果都很差。直到我静下心来，硬着头皮去啃 SQLite 那全是英文的官方文档，理解了“回滚日志”和“预写日志”的区别后，才找到了 WAL 模式这个“金钥匙”。当加上那一行配置代码，看到两个程序终于流畅并发运行的那一刻，那种如释重负的成就感，比打通任何游戏都要强烈。这让我深刻意识到，编程的核心能力不是背诵语法，而是查阅文档和解决未知问题的能力。

此外，这次项目也让我对“全栈”有了初体验。以前我觉得 WinForms 很老土，只喜欢 Web 开发。但这次为了实现毫秒级的硬件控制，我发现 WinForms 依然是王者；而为了实现远程访问，Web 又是不可或缺的。技术没有高下之分，只有场景之别。 懂得根据需求选择最合适的技术栈，才是一个成熟开发者的标志。

虽然这个系统现在看起来界面还不够炫酷，功能也还有待完善（比如还缺少用户权限管理模块），但它是我一行一行代码敲出来、一个一个坑踩过来的。它让我看到了代码连接物理世界（硬件）和数字世界（Web）的可能性，也更加坚定了我未来投身于计算机软件行业的决心。

[参考文献]

[1]高宝东.基于 C#.NET Core 技术与三菱 PLC 的通信方式研究[J].黑龙江科学,2024,15(08):95-97+101.

[2]董荣伟.基于 C# Winform 的牲畜养殖环境智能监控软件设计[J].信息与电脑(理论版),2023,35(16):102-105.

[3]张华贵,刘泽华,李伟.基于 WinForm 与 C#的 Oracle 数据库智能备份软件研发[J].电脑知识与技术,2025,21(21):45-48.DOI:10.14004/j.cnki.ckt.2025.1044.

[4]韦薇.基于 Python 与 WinForm 的书法课件系统的设计与实现[D].北京交通大学,2016.

[5]彭芳策.基于 ASP.NET 技术的烟叶仓储管理系统的设计与实现[J].兴义民族师范学院学报,2025,(05):119-124.

[6]陈印,邓林.ASP.NET 项目中 Excel 数据导入导出应用研究[J].内江科技,2025,46(08):49-50+115.

[7]唐毅,陆媚.基于 ASP.NET Core 的企业岗位竞聘投票与综合考评系统设计[J].现代信息科技,2025,9(14):113-117.DOI:10.19850/j.cnki.2096-4706.2025.14.021.

[8]支易.基于 ASP.NET MVC 的高职院校网站的设计与实现[J].软件,2025,46(06):149-151.

[9]高美蓉.基于.NET 的科研档案管理系统设计与实现[J].信息与电脑,2025,37(02):116-118.

[10]高美蓉.基于.NET 的科研档案管理系统设计与实现[J].信息与电脑,2025,37(02):116-118.